In [4]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

In [1]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import lightgbm as lgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_scor
```

In [7]:
```
df = pd.read_csv('/content/drive/MyDrive/Academics/colab/dataset.csv')
```

In [8]:
```
display(df.columns)
```

```
Index(['timestamp', 'pm25_ugm3', 'pm10_ugm3', 'no_ugm3', 'no2_ugm3', 'nox_pp
b',
       'nh3_ugm3', 'so2_ugm3', 'co_mgm3', 'ozone_ugm3', 'at_c', 'rh_pct',
       'ws_ms', 'wd_deg', 'rf_mm', 'tot_rf_mm', 'sr_wm2', 'state', 'city',
       'station', 'station_id', 'latitude', 'longitude', 'era5_sw_down_wm2',
       'era5_dewpoint_k', 'era5_pressure_pa', 'era5_temp_k',
       'era5_cloud_cover', 'era5_precip_m', 'era5_u10_ms', 'era5_v10_ms',
       'era5_temp_c', 'era5_dewpoint_c', 'era5_pressure_hpa', 'era5_precip_m
m',
       'era5_wind_speed_ms', 'era5_wind_dir_deg', 'era5_vpd_kpa',
       'era5_rh_pct', 'solar_altitude_deg', 'solar_zenith_deg', 'cos_zenit
h'],
      dtype='object')
```

In [9]:
```
display(df.head())
```

	timestamp	pm25_ugm3	pm10_ugm3	no_ugm3	no2_ugm3	nox_ppb	nh3_ugm3	so2
0	2023-01-01 07:00:00	19.0850	38.7950	14.3825	4.9475	14.3225	10.1550	
1	2023-01-01 08:00:00	19.1350	38.8975	9.1650	13.3700	14.5650	10.9125	
2	2023-01-01 09:00:00	19.1250	38.8800	3.1075	14.1600	10.0575	8.3875	
3	2023-01-01 10:00:00	19.1500	38.9275	1.5975	7.2875	5.1775	5.2475	
4	2023-01-01 11:00:00	19.1675	38.9600	1.4775	4.5375	3.6150	3.9950	

5 rows × 42 columns

In [11]:

```
#df.drop(columns=['latitude', 'longitude', 'station_id', 'station', 'state',
from sklearn.preprocessing import LabelEncoder


target_encoders = {}

for col in ['station_id', 'station', 'state', 'city']:
    if col in df.columns:
        target_encoders[col] = df.groupby(col)['sr_wm2'].mean().to_dict()

# Step 2: Apply encoding to entire dataset
for col in ['station_id', 'station', 'state', 'city']:
    if col in df.columns:
        df[f'{col}_encoded'] = df[col].map(target_encoders[col])
        # Fill any missing values (unseen categories) with global mean
        df[f'{col}_encoded'].fillna(df['sr_wm2'].mean(), inplace=True)


float_cols = df.select_dtypes(include=['float64']).columns
df[float_cols] = df[float_cols].astype('float32')

df['timestamp'] = pd.to_datetime(df['timestamp'])

print(df.info())
```

```
/tmp/ipykernel_23367/567494980.py:16: FutureWarning: A value is trying to be
set on a copy of a DataFrame or Series through chained assignment using an in
place method.
The behavior will change in pandas 3.0. This inplace method will never work b
ecause the intermediate object on which we are setting values always behaves
as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.
method({col: value}, inplace=True)' or df[col] = df[col].method(value) instea
d, to perform the operation inplace on the original object.


  df[f'{col}_encoded'].fillna(df['sr_wm2'].mean(), inplace=True)
/tmp/ipykernel_23367/567494980.py:16: FutureWarning: A value is trying to be
set on a copy of a DataFrame or Series through chained assignment using an in
place method.
The behavior will change in pandas 3.0. This inplace method will never work b
ecause the intermediate object on which we are setting values always behaves
as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.
method({col: value}, inplace=True)' or df[col] = df[col].method(value) instea
d, to perform the operation inplace on the original object.


  df[f'{col}_encoded'].fillna(df['sr_wm2'].mean(), inplace=True)
/tmp/ipykernel_23367/567494980.py:16: FutureWarning: A value is trying to be
set on a copy of a DataFrame or Series through chained assignment using an in
place method.
The behavior will change in pandas 3.0. This inplace method will never work b
ecause the intermediate object on which we are setting values always behaves
as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.
method({col: value}, inplace=True)' or df[col] = df[col].method(value) instea
d, to perform the operation inplace on the original object.


  df[f'{col}_encoded'].fillna(df['sr_wm2'].mean(), inplace=True)
/tmp/ipykernel_23367/567494980.py:16: FutureWarning: A value is trying to be
set on a copy of a DataFrame or Series through chained assignment using an in
place method.
The behavior will change in pandas 3.0. This inplace method will never work b
ecause the intermediate object on which we are setting values always behaves
as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.
method({col: value}, inplace=True)' or df[col] = df[col].method(value) instea
d, to perform the operation inplace on the original object.


  df[f'{col}_encoded'].fillna(df['sr_wm2'].mean(), inplace=True)
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2549291 entries, 0 to 2549290
Data columns (total 46 columns):
 #   Column              Dtype
---  ------              -----
 0   timestamp           datetime64[ns]
 1   pm25_ugm3           float32
 2   pm10_ugm3           float32
 3   no_ugm3             float32
 4   no2_ugm3            float32
 5   nox_ppb             float32
 6   nh3_ugm3            float32
 7   so2_ugm3            float32
 8   co_mgm3             float32
 9   ozone_ugm3          float32
 10  at_c                float32
 11  rh_pct              float32
 12  ws_ms               float32
 13  wd_deg              float32
 14  rf_mm               float32
 15  tot_rf_mm           float32
 16  sr_wm2              float32
 17  state               object
 18  city                object
 19  station             object
 20  station_id          object
 21  latitude            float32
 22  longitude           float32
 23  era5_sw_down_wm2    float32
 24  era5_dewpoint_k     float32
 25  era5_pressure_pa    float32
 26  era5_temp_k         float32
 27  era5_cloud_cover    float32
 28  era5_precip_m       float32
 29  era5_u10_ms         float32
 30  era5_v10_ms         float32
 31  era5_temp_c         float32
 32  era5_dewpoint_c     float32
 33  era5_pressure_hpa   float32
 34  era5_precip_mm      float32
 35  era5_wind_speed_ms  float32
 36  era5_wind_dir_deg   float32
 37  era5_vpd_kpa        float32
 38  era5_rh_pct         float32
 39  solar_altitude_deg  float32
 40  solar_zenith_deg    float32
 41  cos_zenith          float32
 42  station_id_encoded  float32
 43  station_encoded     float32
 44  state_encoded       float32
 45  city_encoded        float32
dtypes: datetime64[ns](1), float32(41), object(4)
memory usage: 496.0+ MB
None
```

In [12]:
```python
# Make sure rows are in chronological order
df = df.sort_values("timestamp").reset_index(drop=True)

# Feature engineering
df["year"] = df["timestamp"].dt.year
df["month"] = df["timestamp"].dt.month
df["day"] = df["timestamp"].dt.day
df["hour"] = df["timestamp"].dt.hour

# Drop obvious redundancies
cols_to_drop = [
    'timestamp',                    # Already extracted as year/month/day/ho
    'at_c', 'rh_pct', 'ws_ms', 'wd_deg', 'rf_mm', 'tot_rf_mm',  # Duplicated
    'station_id', 'station', 'state', 'city',  # Keep encoded versions only
    'era5_temp_k', 'era5_dewpoint_k', 'era5_pressure_pa',  # Keep _c and _hp
    'era5_u10_ms', 'era5_v10_ms',   # Use era5_wind_speed_ms instead
    'era5_precip_m',                # Keep era5_precip_mm
    'solar_altitude_deg',           # Keep solar_zenith_deg (more informativ
    'era5_sw_down_wm2',             # Target leakage!
    "sr_wm2",'era5_temp_c',
]
```

In [21]:
```python
# Check for missing values
missing_data = df.isnull().sum()
print("Missing values per column:\n", missing_data[missing_data > 0])

# Summary statistics of numerical columns

# Check temporal coverage
print(f"Data ranges from {df['timestamp'].min()} to {df['timestamp'].max()}"
```

```
Missing values per column:
 Series([], dtype: int64)
Data ranges from 2023-01-01 07:00:00 to 2025-12-31 18:00:00
```

In [13]:
```python
# Step 3: Keep latitude/longitude as numeric, drop original categorical colu
X = df.drop(columns=cols_to_drop)
y = df["sr_wm2"]

# Your existing train/test split remains the same
split_idx = int(len(df) * 0.8)
X_train = X.iloc[:split_idx].copy()
X_test = X.iloc[split_idx:].copy()
y_train = y.iloc[:split_idx].copy()
y_test = y.iloc[split_idx:].copy()

print("Encoded features:")
print(X_train.columns.tolist())
print(f"\nX_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
```

```
Encoded features:
['pm25_ugm3', 'pm10_ugm3', 'no_ugm3', 'no2_ugm3', 'nox_ppb', 'nh3_ugm3', 'so2
_ugm3', 'co_mgm3', 'ozone_ugm3', 'latitude', 'longitude', 'era5_cloud_cover',
'era5_dewpoint_c', 'era5_pressure_hpa', 'era5_precip_mm', 'era5_wind_speed_m
s', 'era5_wind_dir_deg', 'era5_vpd_kpa', 'era5_rh_pct', 'solar_zenith_deg',
'cos_zenith', 'station_id_encoded', 'station_encoded', 'state_encoded', 'city
_encoded', 'year', 'month', 'day', 'hour']

X_train shape: (2039432, 29)
X_test shape: (509859, 29)
```

In [ ]:
```python
from cuml.ensemble import RandomForestRegressor

rf = RandomForestRegressor(
    n_estimators=100,
    max_depth=10,
    random_state=42,
)

rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)

print("Random Forest Results:")
print("MAE:", mean_absolute_error(y_test, y_pred_rf))
print("R2:", r2_score(y_test, y_pred_rf))
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
/tmp/ipykernel_23367/2653955997.py in <cell line: 0>()
----> 1 from cuml.ensemble import RandomForestRegressor
      2 from xgboost import XGBRegressor
      3
      4 rf = RandomForestRegressor(
      5     n_estimators=100,

ModuleNotFoundError: No module named 'cuml'

---------------------------------------------------------------------------
NOTE: If your import is failing due to a missing package, you can
manually install dependencies using either !pip or !apt.

To view examples of installing some common dependencies, click the
"Open Examples" button below.
---------------------------------------------------------------------------
```

In [ ]:
```python
from cuml.ensemble import RandomForestRegressor
from sklearn.model_selection import TimeSeriesSplit, ParameterSampler
from sklearn.metrics import mean_absolute_error, r2_score
import numpy as np

# ----------------------------------------------------------------------
# Random Forest hyperparameter tuning
# ----------------------------------------------------------------------

param_dist = {
```

```
    "n_estimators": [100, 200, 400, 600, 800],
    "max_depth": [5, 10, 15, 20, None],
    "max_features": ["sqrt", "log2", 0.5, 0.7, 1.0],
    "min_samples_split": [2, 5, 10, 20],
    "min_samples_leaf": [1, 2, 4, 8],
    "bootstrap": [True, False],
}

tscv = TimeSeriesSplit(n_splits=4)

best_score = -np.inf
best_params = None
best_model = None

# Try a limited number of random combinations
n_iter = 25

for params in ParameterSampler(param_dist, n_iter=n_iter, random_state=42):
    cv_scores = []

    for train_idx, val_idx in tscv.split(X_train):
        X_tr, X_val = X_train.iloc[train_idx], X_train.iloc[val_idx]
        y_tr, y_val = y_train.iloc[train_idx], y_train.iloc[val_idx]

        model = RandomForestRegressor(
            random_state=42,
            n_streams=1,
            **params
        )

        model.fit(X_tr, y_tr)
        val_pred = model.predict(X_val)
        score = r2_score(y_val, val_pred)
        cv_scores.append(score)

    mean_cv_score = np.mean(cv_scores)

    if mean_cv_score > best_score:
        best_score = mean_cv_score
        best_params = params

print("Best CV R2:", best_score)
print("Best params:", best_params)

# Refit best model on full training set
best_rf = RandomForestRegressor(
    random_state=42,
    n_streams=1,
    **best_params
)

best_rf.fit(X_train, y_train)

# Evaluate on hold-out test set
y_pred_rf = best_rf.predict(X_test)
```

```
print("\nRandom Forest Results:")
print("MAE:", mean_absolute_error(y_test, y_pred_rf))
print("R2:", r2_score(y_test, y_pred_rf))
```

In [15]:
```
from xgboost import XGBRegressor

xgb = XGBRegressor(
    n_estimators=300,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    tree_method='hist',     # Enalbe GPU acceleration
    random_state=42,
    n_jobs=-1
)

xgb.fit(X_train, y_train)

y_pred_xgb = xgb.predict(X_test)

print("\nXGBoost Results:")
print("MAE:", mean_absolute_error(y_test, y_pred_xgb))
print("R2:", r2_score(y_test, y_pred_xgb))
# old: .53
```

```
XGBoost Results:
MAE: 73.59874725341797
R2: 0.6284855604171753
```

In [ ]:
```
from xgboost import XGBRegressor
from sklearn.model_selection import TimeSeriesSplit, RandomizedSearchCV
from sklearn.metrics import mean_absolute_error, r2_score
import numpy as np

# If X_train, X_test, y_train, y_test are already prepared, use them directl
# Otherwise make sure you've done the chronological split first.

xgb = XGBRegressor(
    objective="reg:squarederror",
    tree_method="hist",
    random_state=42,
    n_jobs=-1
)

param_dist = {
    "n_estimators": [200, 400, 600, 800, 1000],
    "learning_rate": [0.01, 0.03, 0.05, 0.1],
    "max_depth": [3, 4, 5, 6, 8, 10],
    "min_child_weight": [1, 3, 5, 7],
    "subsample": [0.6, 0.8, 1.0],
    "colsample_bytree": [0.6, 0.8, 1.0],
    "gamma": [0, 0.1, 0.2, 0.5, 1.0],
    "reg_alpha": [0.0, 0.01, 0.1, 1.0],
    "reg_lambda": [0.5, 1.0, 2.0, 5.0]
}
```

```
tscv = TimeSeriesSplit(n_splits=4)

search = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_dist,
    n_iter=30,
    scoring="r2",
    cv=tscv,
    verbose=2,
    random_state=42,
    n_jobs=-1
)

search.fit(X_train, y_train)

print("Best CV R2:", search.best_score_)
print("Best params:", search.best_params_)

best_xgb = search.best_estimator_

y_pred_xgb = best_xgb.predict(X_test)

print("\nXGBoost Results:")
print("MAE:", mean_absolute_error(y_test, y_pred_xgb))
print("R2:", r2_score(y_test, y_pred_xgb))
```

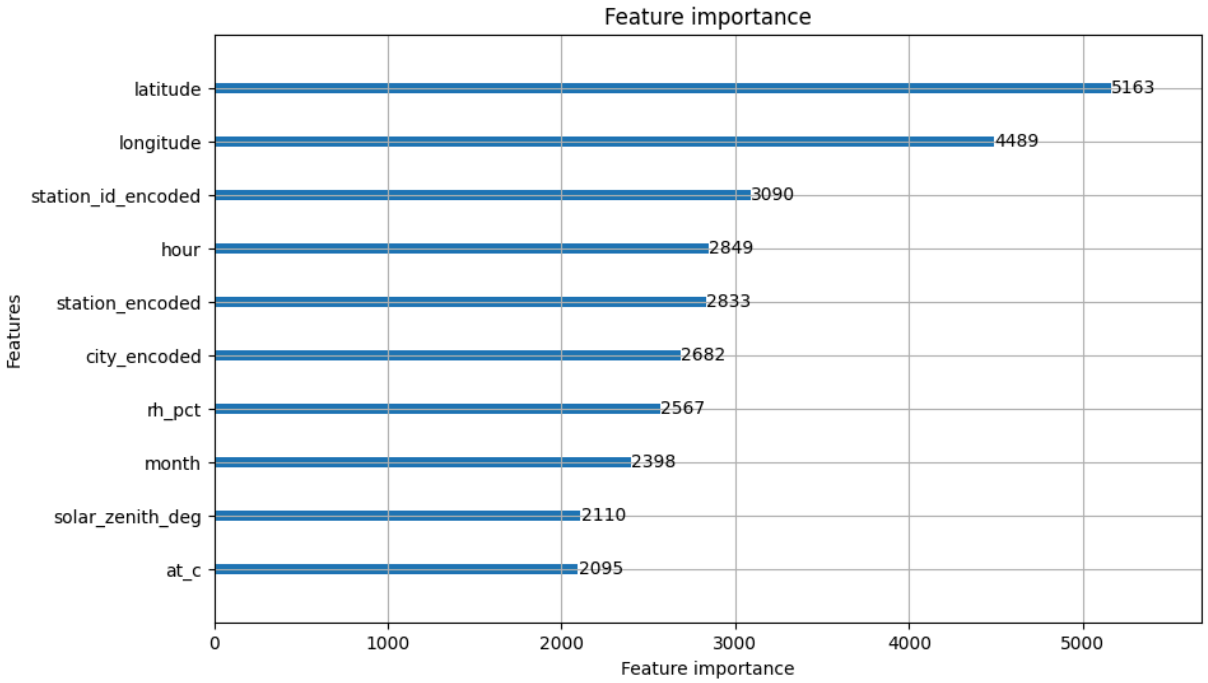
In [14]:

```
train_data = lgb.Dataset(X_train, label=y_train)
test_data = lgb.Dataset(X_test, label=y_test, reference=train_data)

params = {
    'objective': 'regression',
    'metric': 'rmse',
    'boosting_type': 'gbdt',
    'device': 'gpu',           # Enable GPU
    'gpu_platform_id': 0,
    'gpu_device_id': 0,
    'learning_rate': 0.05,
    'num_leaves': 63,
    'feature_fraction': 0.8,
}

model = lgb.train(params, train_data, num_boost_round=1000, valid_sets=[trai

# 7. Predictions & Evaluation
y_pred = model.predict(X_test, num_iteration=model.best_iteration)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("\n--- Model Performance ---")
print(f"RMSE: {rmse:.2f} W/m²")
print(f"MAE:  {mae:.2f} W/m²")
print(f"R² Score: {r2:.4f}")
```

```
# 8. Feature Importance
print("\nTop 10 Most Important Features:")
lgb.plot_importance(model, max_num_features=10, figsize=(10, 6))
```

```
[LightGBM] [Info] This is the GPU trainer!!
[LightGBM] [Info] Total Bins 9646
[LightGBM] [Info] Number of data points in the train set: 2039432, number of
used features: 43
[LightGBM] [Info] Using requested OpenCL platform 0 device 0
[LightGBM] [Info] Using GPU Device: Tesla T4, Vendor: NVIDIA Corporation
[LightGBM] [Info] Compiling OpenCL Kernel with 256 bins...
[LightGBM] [Info] GPU programs have been built
[LightGBM] [Info] Size of histogram bin entry: 8
[LightGBM] [Info] 39 dense feature groups (77.80 MB) transferred to GPU in 0.
099677 secs. 1 sparse feature groups
[LightGBM] [Info] Start training from score 210.234281

--- Model Performance ---
RMSE: 118.56 W/m²
MAE:  65.73 W/m²
R² Score: 0.6756

Top 10 Most Important Features:
```

Out[14]: <Axes: title={'center': 'Feature importance'}, xlabel='Feature importance', ylabel='Features'>



In [ ]:
```
# Hyperparameter search (time-aware) for LightGBM to maximize R2
from sklearn.model_selection import TimeSeriesSplit, RandomizedSearchCV
from sklearn.metrics import r2_score, mean_absolute_error
import lightgbm as lgb
import numpy as np
import pandas as pd

# 1) Time split (80% train, 20% test)
split_idx = int(len(df) * 0.8)
```

```
train_df = df.iloc[:split_idx].copy().reset_index(drop=True)
test_df  = df.iloc[split_idx:].copy().reset_index(drop=True)

# 2) Fit target-encoding on train only (prevent leakage)
cat_cols = [c for c in ['station_id','station','state','city'] if c in df.co
for c in cat_cols:
    means = train_df.groupby(c)['sr_wm2'].mean()
    train_df[f'{c}_enc'] = train_df[c].map(means).fillna(train_df['sr_wm2'].
    test_df[f'{c}_enc']  = test_df[c].map(means).fillna(train_df['sr_wm2'].m

# 3) Build X/y (drop leak columns)
drop_cols = ['sr_wm2', 'era5_sw_down_wm2', 'timestamp'] + cat_cols
X_train = train_df.drop(columns=[c for c in drop_cols if c in train_df.colum
X_test  = test_df.drop(columns=[c for c in drop_cols if c in test_df.columns
y_train = train_df['sr_wm2']
y_test  = test_df['sr_wm2']

# 4) LightGBM estimator (sklearn API)
est = lgb.LGBMRegressor(objective='regression', random_state=42, n_jobs=-1)

# 5) Parameter search space (common, compact)
param_dist = {
    'num_leaves': [31, 63, 127, 255],
    'max_depth': [-1, 6, 10, 15],
    'learning_rate': [0.01, 0.03, 0.05, 0.1],
    'n_estimators': [100, 300, 1000],
    'min_child_samples': [10, 20, 50, 100],
    'subsample': [0.6, 0.8, 1.0],
    'colsample_bytree': [0.6, 0.8, 1.0],
    'reg_alpha': [0.0, 0.1, 1.0],
    'reg_lambda': [0.0, 0.1, 1.0]
}

# 6) Time-series CV and randomized search
tscv = TimeSeriesSplit(n_splits=4)  # preserves time order
search = RandomizedSearchCV(
    est,
    param_distributions=param_dist,
    n_iter=30,
    scoring='r2',
    cv=tscv,
    random_state=42,
    n_jobs=-1,
    verbose=2
)

# Run search (may take time)
search.fit(X_train, y_train)

# 7) Results
print("Best CV R2:", search.best_score_)
print("Best params:", search.best_params_)

# Evaluate on hold-out test set
best = search.best_estimator_
y_pred = best.predict(X_test)
```

```
print("Test R2:", r2_score(y_test, y_pred))
print("Test MAE:", mean_absolute_error(y_test, y_pred))

# Optional: Save best model
# import joblib
# joblib.dump(best, "lgbm_best.pkl")
```

```
Fitting 4 folds for each of 30 candidates, totalling 120 fits
/usr/local/lib/python3.12/dist-packages/joblib/externals/loky/process_executo
r.py:782: UserWarning: A worker stopped while some jobs were given to the exe
cutor. This can be caused by a too short worker timeout or by a memory leak.
  warnings.warn(
```

In [11]:

```
from tensorflow import keras
from tensorflow.keras import layers

# Create a shallow neural network
model = keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)),
    layers.Dense(32, activation='relu'),
    layers.Dense(1)  # Output layer for regression
])

# Compile the model
model.compile(
    optimizer='adam',
    loss='mean_squared_error',
    metrics=['mae']
)

# Train the model
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2,
    verbose=2
)

# Test the model
y_pred_nn = model.predict(X_test)

# Evaluation
rmse = np.sqrt(mean_squared_error(y_test, y_pred_nn))
mae = mean_absolute_error(y_test, y_pred_nn)
r2 = r2_score(y_test, y_pred_nn)

print("\n--- Neural Network Results ---")
print(f"RMSE: {rmse:.2f} W/m²")
print(f"MAE:  {mae:.2f} W/m²")
print(f"R² Score: {r2:.4f}")

# Plot training history
plt.figure(figsize=(12, 4))
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'], label='Train Loss')
```

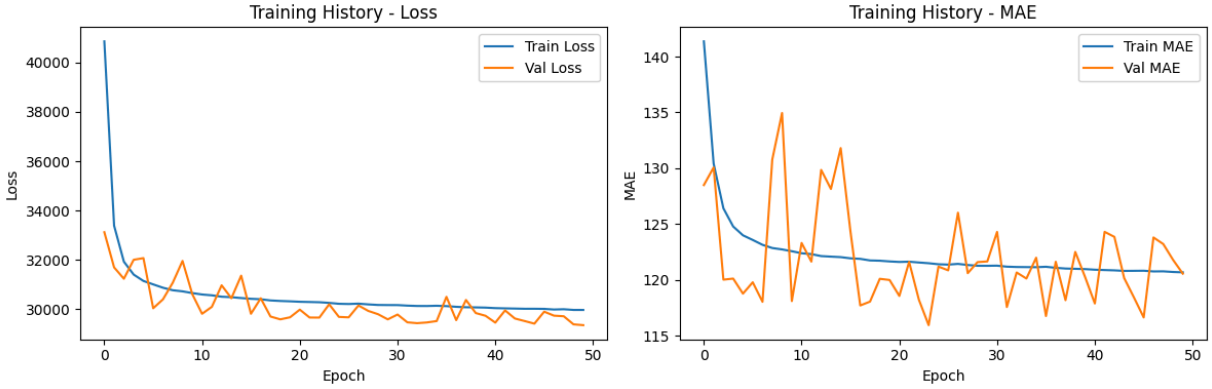
```
plt.plot(history.history['val_loss'], label='Val Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.title('Training History - Loss')

plt.subplot(1, 2, 2)
plt.plot(history.history['mae'], label='Train MAE')
plt.plot(history.history['val_mae'], label='Val MAE')
plt.xlabel('Epoch')
plt.ylabel('MAE')
plt.legend()
plt.title('Training History - MAE')
plt.tight_layout()
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: U
serWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. Whe
n using Sequential models, prefer using an `Input(shape)` object as the first
layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Epoch 1/50
50986/50986 - 166s - 3ms/step - loss: 40862.6289 - mae: 141.3643 - val_loss:
33120.2773 - val_mae: 128.4962
Epoch 2/50
50986/50986 - 160s - 3ms/step - loss: 33384.4062 - mae: 130.4195 - val_loss:
31692.3574 - val_mae: 130.0716
Epoch 3/50
50986/50986 - 202s - 4ms/step - loss: 31925.9062 - mae: 126.4139 - val_loss:
31231.7812 - val_mae: 120.0265
Epoch 4/50
50986/50986 - 160s - 3ms/step - loss: 31400.8242 - mae: 124.7877 - val_loss:
32001.0566 - val_mae: 120.1219
Epoch 5/50
50986/50986 - 161s - 3ms/step - loss: 31142.3438 - mae: 123.9958 - val_loss:
32068.5547 - val_mae: 118.7731
Epoch 6/50
50986/50986 - 158s - 3ms/step - loss: 31002.7637 - mae: 123.5818 - val_loss:
30034.5059 - val_mae: 119.7904
Epoch 7/50
50986/50986 - 160s - 3ms/step - loss: 30867.1621 - mae: 123.1454 - val_loss:
30395.7480 - val_mae: 118.0417
Epoch 8/50
50986/50986 - 161s - 3ms/step - loss: 30765.4785 - mae: 122.8589 - val_loss:
31071.1191 - val_mae: 130.7808
Epoch 9/50
50986/50986 - 162s - 3ms/step - loss: 30720.5000 - mae: 122.7340 - val_loss:
31954.6152 - val_mae: 134.9471
Epoch 10/50
50986/50986 - 159s - 3ms/step - loss: 30650.5000 - mae: 122.5723 - val_loss:
30606.8535 - val_mae: 118.1093
Epoch 11/50
50986/50986 - 159s - 3ms/step - loss: 30592.6406 - mae: 122.3911 - val_loss:
29809.5508 - val_mae: 123.3113
Epoch 12/50
50986/50986 - 157s - 3ms/step - loss: 30559.3145 - mae: 122.3131 - val_loss:
30092.5137 - val_mae: 121.6424
Epoch 13/50
50986/50986 - 159s - 3ms/step - loss: 30503.0684 - mae: 122.1366 - val_loss:
30965.4297 - val_mae: 129.8530
Epoch 14/50
50986/50986 - 158s - 3ms/step - loss: 30483.2188 - mae: 122.0882 - val_loss:
30441.4531 - val_mae: 128.1479
Epoch 15/50
50986/50986 - 157s - 3ms/step - loss: 30448.6504 - mae: 122.0438 - val_loss:
31356.9395 - val_mae: 131.8092
Epoch 16/50
50986/50986 - 156s - 3ms/step - loss: 30418.5117 - mae: 121.9354 - val_loss:
29810.3965 - val_mae: 124.3306
Epoch 17/50
50986/50986 - 158s - 3ms/step - loss: 30400.3164 - mae: 121.8885 - val_loss:
30438.1797 - val_mae: 117.7147
Epoch 18/50
50986/50986 - 201s - 4ms/step - loss: 30351.3887 - mae: 121.7459 - val_loss:
29699.3789 - val_mae: 118.0607
Epoch 19/50
50986/50986 - 157s - 3ms/step - loss: 30330.0371 - mae: 121.7227 - val_loss:
```

```
29584.3438 - val_mae: 120.1021
Epoch 20/50
50986/50986 - 156s - 3ms/step - loss: 30315.8633 - mae: 121.6569 - val_loss:
29668.2637 - val_mae: 119.9973
Epoch 21/50
50986/50986 - 156s - 3ms/step - loss: 30297.4180 - mae: 121.6112 - val_loss:
29978.5918 - val_mae: 118.5661
Epoch 22/50
50986/50986 - 156s - 3ms/step - loss: 30288.3008 - mae: 121.6343 - val_loss:
29658.5352 - val_mae: 121.6211
Epoch 23/50
50986/50986 - 155s - 3ms/step - loss: 30278.6270 - mae: 121.5620 - val_loss:
29655.6094 - val_mae: 118.2302
Epoch 24/50
50986/50986 - 156s - 3ms/step - loss: 30252.0684 - mae: 121.4966 - val_loss:
30195.2383 - val_mae: 115.9511
Epoch 25/50
50986/50986 - 156s - 3ms/step - loss: 30215.5918 - mae: 121.4040 - val_loss:
29683.4453 - val_mae: 121.1948
Epoch 26/50
50986/50986 - 205s - 4ms/step - loss: 30206.9316 - mae: 121.3731 - val_loss:
29666.3301 - val_mae: 120.8602
Epoch 27/50
50986/50986 - 156s - 3ms/step - loss: 30221.2168 - mae: 121.4382 - val_loss:
30147.1113 - val_mae: 126.0201
Epoch 28/50
50986/50986 - 154s - 3ms/step - loss: 30191.0137 - mae: 121.3379 - val_loss:
29926.7207 - val_mae: 120.6058
Epoch 29/50
50986/50986 - 155s - 3ms/step - loss: 30168.2578 - mae: 121.2702 - val_loss:
29793.6074 - val_mae: 121.5971
Epoch 30/50
50986/50986 - 154s - 3ms/step - loss: 30164.1465 - mae: 121.2673 - val_loss:
29583.8691 - val_mae: 121.6559
Epoch 31/50
50986/50986 - 155s - 3ms/step - loss: 30162.8652 - mae: 121.2784 - val_loss:
29779.9746 - val_mae: 124.3028
Epoch 32/50
50986/50986 - 156s - 3ms/step - loss: 30140.9258 - mae: 121.1881 - val_loss:
29464.1094 - val_mae: 117.5730
Epoch 33/50
50986/50986 - 201s - 4ms/step - loss: 30127.4375 - mae: 121.1603 - val_loss:
29429.1465 - val_mae: 120.6686
Epoch 34/50
50986/50986 - 155s - 3ms/step - loss: 30125.8672 - mae: 121.1502 - val_loss:
29458.7520 - val_mae: 120.1262
Epoch 35/50
50986/50986 - 155s - 3ms/step - loss: 30135.0137 - mae: 121.1433 - val_loss:
29514.8457 - val_mae: 121.9911
Epoch 36/50
50986/50986 - 154s - 3ms/step - loss: 30121.2988 - mae: 121.1796 - val_loss:
30497.4473 - val_mae: 116.7657
Epoch 37/50
50986/50986 - 155s - 3ms/step - loss: 30092.5527 - mae: 121.0861 - val_loss:
29552.3848 - val_mae: 121.6317
Epoch 38/50
```

```
50986/50986 - 154s - 3ms/step - loss: 30075.6094 - mae: 121.0219 - val_loss:
30370.0898 - val_mae: 118.1873
Epoch 39/50
50986/50986 - 156s - 3ms/step - loss: 30069.4062 - mae: 120.9977 - val_loss:
29839.6172 - val_mae: 122.5099
Epoch 40/50
50986/50986 - 155s - 3ms/step - loss: 30060.4023 - mae: 120.9720 - val_loss:
29729.0098 - val_mae: 120.2655
Epoch 41/50
50986/50986 - 155s - 3ms/step - loss: 30039.7773 - mae: 120.9165 - val_loss:
29451.6816 - val_mae: 117.8913
Epoch 42/50
50986/50986 - 156s - 3ms/step - loss: 30030.0898 - mae: 120.8864 - val_loss:
29946.8945 - val_mae: 124.3073
Epoch 43/50
50986/50986 - 154s - 3ms/step - loss: 30021.8926 - mae: 120.8583 - val_loss:
29618.8691 - val_mae: 123.8588
Epoch 44/50
50986/50986 - 155s - 3ms/step - loss: 30013.1836 - mae: 120.8098 - val_loss:
29515.6562 - val_mae: 120.1688
Epoch 45/50
50986/50986 - 154s - 3ms/step - loss: 30013.6152 - mae: 120.8164 - val_loss:
29409.9883 - val_mae: 118.4339
Epoch 46/50
50986/50986 - 156s - 3ms/step - loss: 30008.9980 - mae: 120.8245 - val_loss:
29896.2773 - val_mae: 116.6403
Epoch 47/50
50986/50986 - 154s - 3ms/step - loss: 29983.6406 - mae: 120.7634 - val_loss:
29734.1680 - val_mae: 123.8047
Epoch 48/50
50986/50986 - 156s - 3ms/step - loss: 29994.6289 - mae: 120.7712 - val_loss:
29710.2070 - val_mae: 123.2148
Epoch 49/50
50986/50986 - 154s - 3ms/step - loss: 29966.4883 - mae: 120.7135 - val_loss:
29381.8008 - val_mae: 121.8018
Epoch 50/50
50986/50986 - 157s - 3ms/step - loss: 29966.0000 - mae: 120.6832 - val_loss:
29348.4570 - val_mae: 120.5634
15934/15934 ━━━━━━━━━━━━━━━━━━━━ 26s 2ms/step

--- Neural Network Results ---
RMSE: 172.12 W/m²
MAE:  120.92 W/m²
R² Score: 0.3373
```

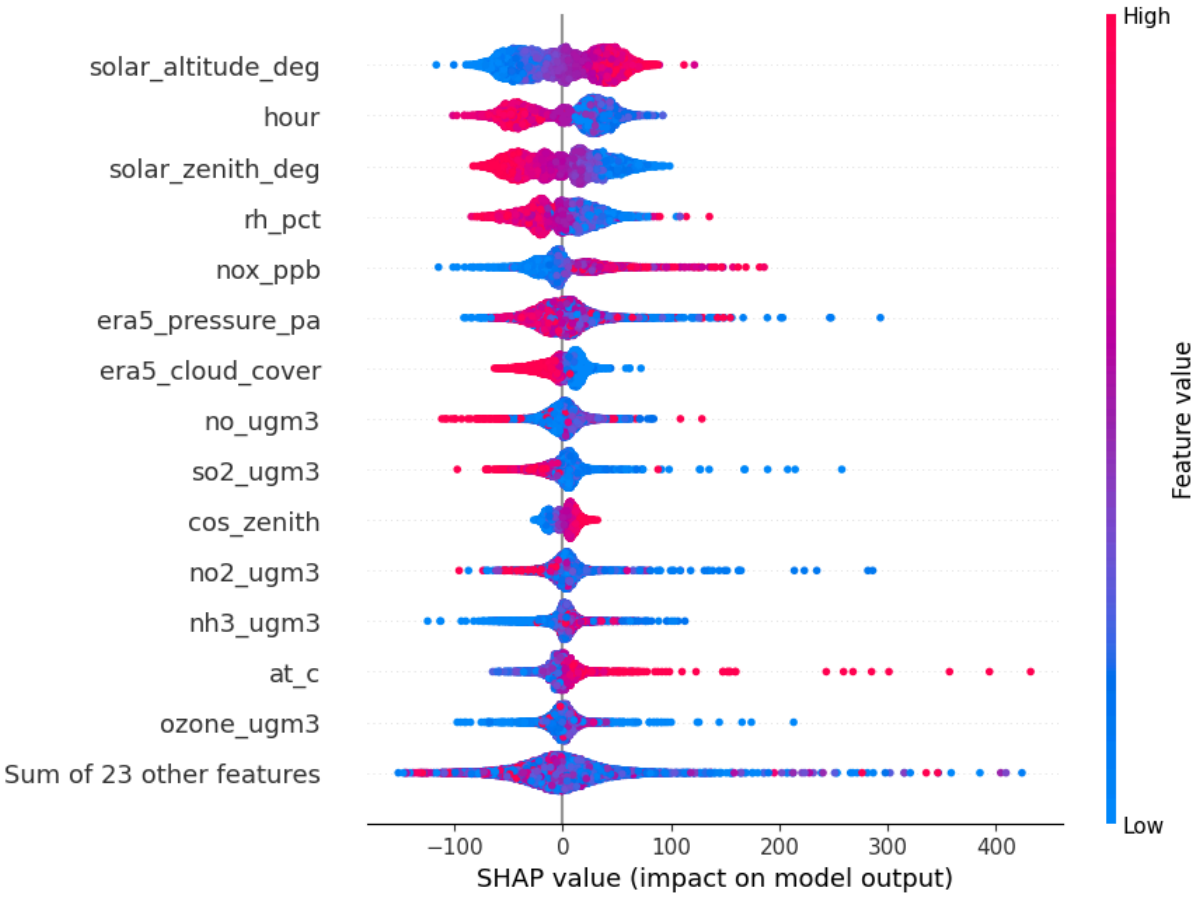


In [ ]:

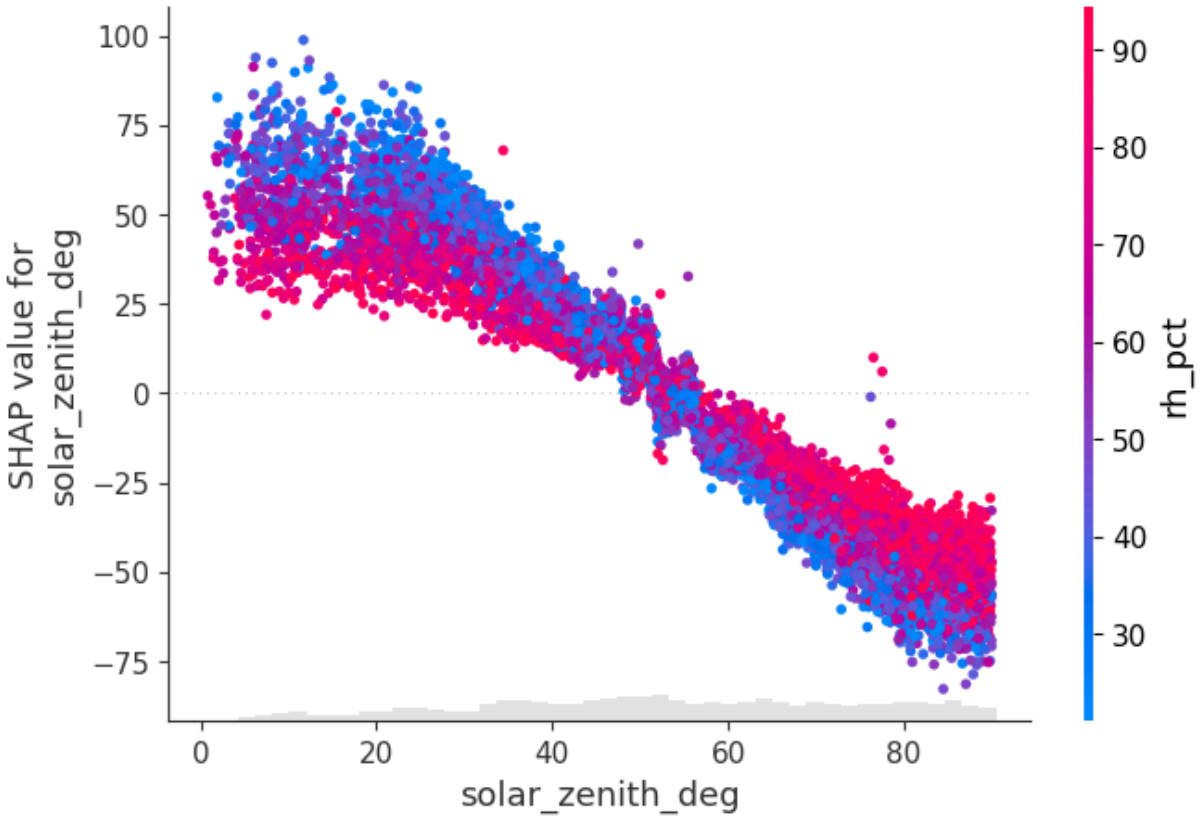
```
import shap

X_shap = X_test.sample(n=10000, random_state=42)

explainer = shap.TreeExplainer(model)

print("Calculating SHAP values...")
shap_values = explainer(X_shap)

# PLOT 1: FORCE PLOT (Single Prediction)
shap.initjs()

print("Generating Force Plot...")
shap.plots.force(shap_values[0])


# PLOT 2: SUMMARY PLOT (Overall Importance)
print("Generating Summary Plot...")
# A beeswarm plot shows both the rank of importance AND the directional impa
shap.plots.beeswarm(shap_values, max_display=15)

# PLOT 3: DEPENDENCE PLOT (Feature Effect)
print("Generating Dependence Plot...")
shap.plots.scatter(shap_values[:, "solar_zenith_deg"], color=shap_values)
```

```
Calculating SHAP values...
```

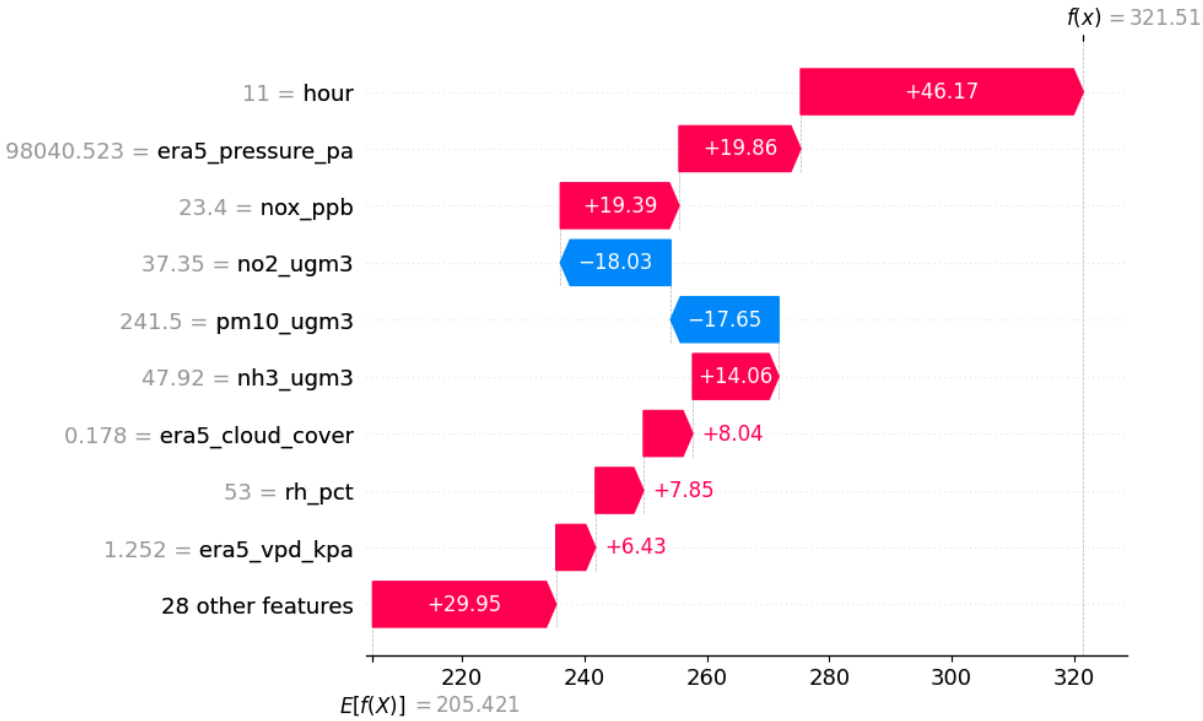

```
Generating Force Plot...
Generating Summary Plot...
```




```
Generating Dependence Plot...
```



In [19]:
```
shap.plots.waterfall(shap_values[0])
```


f(x) = 321.51
11 = hour
98040.523 = era5_pressure_pa
23.4 = nox_ppb
37.35 = no2_ugm3
241.5 = pm10_ugm3
47.92 = nh3_ugm3
0.178 = era5_cloud_cover
53 = rh_pct
1.252 = era5_vpd_kpa
28 other features
+46.17
+19.86
+19.39
−18.03
−17.65
+14.06
+8.04
+7.85
+6.43
+29.95
220
240
260
280
300
320
E[f(X)] = 205.421